PROJECT REPORT

On

HydroFlow V2: Flood Outlook and Humanitarian Planning for Bago Region, Myanmar

Submitted

In partial fulfilment

For the award of the Degree of

[Degree programme]

[Institution]

Guided By: [Name of the project guide]

Submitted By:

[Name of the candidate]

[Institution and address]

November 2026

Acknowledgment

The author thanks the project guide for supervision, technical comment, and the time given to reviewing this work. The teaching staff and the computing facilities of the institution made the prototype possible.

Colleagues who discussed the rainfall records, the township maps, and the limits of an academic prototype improved the clarity of the result. Families and friends provided the patience required to finish the book.

The names of the guide, the candidate, the degree, and the institution are left in brackets on the cover because they are not recorded in the HydroFlow V2 source tree. They should be written in before submission.

Abstract

Flood response in Bago Region needs an early view of where heavy rain and rising river discharge may coincide with communities that later appear in relief records. HydroFlow V2 is an academic prototype for twenty-three townships. It joins a flood-occurrence score, a short assistance illustration, and a camera count of relief goods.

The served flood classifier is a histogram gradient boosting model with fifteen hydrometeorological features. The candidate saved in the model file is hgb-depth3. Its decision threshold of 0.05 is chosen on a chronological validation block, and the final test block is left untouched. On 8,052 test township-days, of which 11 are flood days, the model records precision 0.0113, recall 0.6364, F1 0.0222, and precision-recall area 0.0085.

A class-weighted logistic regression uses the same fifteen features and the same split. It is retained as an interpretable baseline. The prediction service loads the histogram model, not the logistic file. A rule that always predicts no flood is accurate on this sparse test block, so accuracy is reported beside precision and recall and is not treated as evidence of skill.

The interface has a home view, a flood-outlook map, a resource-allocation view, and an inventory view. Assistance counts use fixed planning ratios. Routes are drawn from township centroids. The prototype demonstrates an end-to-end workflow. It is not validated for public warnings, caseload estimates, or dispatch.

Table of Contents

Front matter is numbered in roman numerals. Chapters are numbered in arabic numerals from 1. The Page column uses those printed numbers.

S.No	Chapter	Title	Page
1	—	Acknowledgment	i
2	—	Abstract	i
3	—	Table of Contents	ii
4	**Chapter 1**	**Introduction**	**1**
5	1.1	Purpose of the Project	1
6	1.2	Objectives of the Project	1
7	1.3	Problem Definition	2
8	1.4	Existing System and Limitations	2
9	1.5	Organization of the Project	3
10	**Chapter 2**	**Literature Review**	**3**
11	**Chapter 3**	**Methodology and Techniques**	**4**
12	3.1	Machine-Learning Approaches for Flood Occurrence	5
13	3.1.1	Class-Weighted Logistic Regression (interpretable baseline)	5
14	3.1.2	Histogram Gradient Boosting (served flood model)	5
15	3.2	Application Services and External Data Clients	6
16	3.3	Relief, Census, and Inventory Data Preparation	6
17	3.4	Feature Construction and Numeric Representation	7
18	3.5	Application Database and Processed Data Store	8
19	3.6	Multi-Source Hydrometeorological Inputs	8

20	3.7	Leakage Control and Claim Limits	9
21	3.8	Request Handling and Response Generation	9
22	3.9	Proposed System Workflow	10
23	**Chapter 4**	**System Design and Implementation**	**10**
24	4.1	Overview of the Proposed System	10
25	4.2	Scope of the Project	11
26	4.3	Dataset Description	11
27	4.3.1	Source Data Files	12
28	4.3.2	Data Preprocessing	12
29	4.4	Technical Stack and Configuration	12
30	4.5	System Implementation	13
31	**Chapter 5**	**Results and Discussion**	**14**
32	5.1	Introduction	14
33	5.2	System Output and Functional Results	14
34	5.3	Logistic Regression Baseline versus Served Histogram Gradient Boosting	15
35	5.4	Graphical User Interface	16
36	5.5	System Testing and Model Evaluation	16
37	5.6	Results Analysis	17
38	5.7	Model Selection and Justification	18
39	5.8	Advantages of the Proposed System	18
40	**Chapter 6**	**Conclusion and Future Scope**	**19**
41	6.1	Summary of the Project	19
42	6.2	Limitations	19
43	6.3	Future Extensions	20
44	—	References	20

Chapter 1

Introduction

Bago Region sits on a low floodplain where monsoon rain and river discharge can rise together. A township office that plans relief still has to look across weather, past flood days, population, stock, and the road to the next township. HydroFlow V2 is a single academic workspace for that inspection.

The unit of analysis for the flood model is a township-day. The label is the observed relief flood flag stored with the daily table. The model file states that no synthetic flood rows were added. A positive label means a flood day present in the available relief records. It is not a complete hydrological inventory of every inundated field.

1.1 Purpose of the Project

The purpose of the project is to show a correct path from stored and live hydrometeorological inputs to a township flood score, a short planning illustration, and a count of relief goods. The path is meant for study and demonstration.

The served score comes from a saved scikit-learn histogram gradient boosting classifier. The application loads that file and does not retrain when a user opens the map. A second file, a class-weighted logistic regression, is kept so the same features and the same untouched test block can be read in a linear form.

The purpose stops at the academic prototype. Outputs on the screen are labeled as prototype results. They are not an instruction to issue a public warning or to dispatch a truck.

1.2 Objectives of the Project

The first objective is to train and serve a flood-occurrence model on a chronological split of Bago township-days, with the test block held out from fitting, calibration, and threshold selection.

The second objective is to keep an interpretable logistic baseline on the same fifteen features, so model selection can be explained with precision, recall, F1, and precision-recall area rather than with accuracy alone.

The third objective is to place that score in a working system: a FastAPI service, a React map, a resource-allocation screen, and an inventory screen that can count foods, clothing, kitchenware, and utilities from a photograph or a video.

The fourth objective is to state the limits in the same book as the scores. Rainfall in the training table is reanalysis. River discharge is modeled GloFAS discharge. Impact counts used for planning are illustrations tied to relief records, census context, or the flood score.

1.3 Problem Definition

Given a township and a date, the flood problem is to estimate whether that township-day matches the relief flood flag, using only inputs that can be assembled from rainfall, modeled discharge, temperature, humidity, wind, location, and the day of the year.

The class is rare. The served metadata records 39,534 township-days with complete features and drops 638 rows for missing features. The validation block has 8,030 rows and 167 flood rows. The test block has 8,052 rows and 11 flood rows. A classifier can look accurate by predicting no flood on almost every day.

A second problem sits beside the score. After a high or severe day, a planner still needs a picture of people and households, a three-day package of shelter, food, and water, a route between township centroids, and a count of what is already in store. HydroFlow V2 treats those as linked screens, not as a claim that the counts are a field assessment.

1.4 Existing System and Limitations

Before this prototype, the practical tools were separate. Weather and discharge came from public archives. Flood days and impact counts lived in relief workbooks. Population context lived in the census extract. Stock was a paper or spreadsheet count. A map, a model, and a route were not one workflow.

An operational forecasting chain that uses machine learning still depends on maintained forecasts, exposure, and local verification [1]. A general review of flood prediction with machine learning collects tree ensembles, linear classifiers, and neural networks as separate families [2]. Those systems depend on labels and measurements that this project does not have for every township-day.

The limitation that shapes HydroFlow V2 is the label itself. Relief flags are incomplete, positives are concentrated in time, and the untouched test block contains only eleven flood

days. An earlier twelve-feature logistic repair, kept as a separate study, removed retrospective soil fields so that a future date could be scored, and its flood F1 was about 0.029. That experiment is not the model loaded by the current prediction service. The service loads the fifteen-feature histogram model described in later chapters.

1.5 Organization of the Project

Chapter 2 reviews flood prediction, imbalanced evaluation, leakage, humanitarian planning quantities, and camera detection of relief goods. Chapter 3 describes the two flood classifiers, the application services, the preparation of relief and inventory data, the fifteen numeric features, storage, the external inputs, the controls on leakage and claims, request handling, and the workflow.

Chapter 4 states the scope, the source files, preprocessing, the technical stack, and how the service and the interface are implemented. Chapter 5 reports the untouched test scores, compares the logistic baseline with the served histogram model, describes the screens, and records how testing is organized. Chapter 6 summarizes the work, lists the limits, and names extensions that would require new labels and an independent review.

Chapter 2

Literature Review

Machine learning for flood prediction is reviewed as a family of methods rather than as a single accepted model [2]. Tree ensembles, logistic models, and neural networks all appear in that literature. Random forests are a standard ensemble baseline in the wider machine-learning literature [3]. An earlier comparison sat beside a random-forest score, and this project does not serve a random forest. The served model belongs to the gradient boosting family introduced for additive stagewise fitting [4], in the histogram form provided by scikit-learn [5].

Rainfall-runoff studies have also used long short-term memory networks when long gauged series exist [6]. HydroFlow V2 does not serve a recurrent network. The daily table is a township panel with a sparse relief flag, and the evaluation design in this book is a single chronological holdout rather than a sequence model trained on gauge traces.

Operational flood forecasting with machine learning has been demonstrated where forecasts, exposure, and verification are maintained as a service [1]. Global river discharge

from GloFAS is a modeled field, produced by a hydrological chain forced with meteorological reanalysis, and it is not a reading from a local staff gauge [7]. HydroFlow V2 uses that modeled discharge, together with Open-Meteo rainfall [8] and NASA POWER meteorology for temperature, humidity, and wind [9]. Copernicus elevation is used as map context. It is a surface model and includes canopy and buildings [10]. It is not one of the fifteen model inputs.

Leakage is the use of information that would not be known at the time of prediction [11]. The training script for the served model keeps the saved chronological split, checks that training dates end before validation and that validation ends before the test, and writes a flag that the test block was not used for training, calibration, or the threshold. Rolling rainfall totals are computed inside each township. Discharge inputs are lagged by one day so the discharge features do not copy the same-day value.

When positives are rare, a precision-recall curve is more informative than accuracy or a receiver-operating curve used alone [12]. The test block here has eleven flood days in 8,052 rows. The metadata also records a majority-class accuracy of 0.9986 for a prediction of no flood on every test row. Chapter 5 therefore leads with precision, recall, F1, and precision-recall area.

Humanitarian planning has published minimum quantities for water, food, and shelter in the Sphere handbook [13]. The software multiplies expected people and households by fixed ratios: one shelter kit per household, three food rations per person across three days, and fifteen litres of drinking water per person per day. Those ratios are planning multipliers in the prototype. They are not a field assessment and they are not a claim that a distribution meets the handbook.

Counting relief goods from a camera is a detection problem. The original detection framework locates objects in one pass [14]. The training runs in this project use the Ultralytics implementation [15]. Four detectors are trained separately for foods, clothing, kitchenware, and utilities. The catalog class fund is not given a detector.

Chapter 3

Methodology and Techniques

This chapter describes the methods that produce the flood score and the surrounding workflow. The prediction service loads one flood artifact. A second classifier is trained on the

same rows for comparison. Assistance figures, routes, and inventory counts are separate steps with their own rules.

3.1 Machine-Learning Approaches for Flood Occurrence

Both flood classifiers answer the same question: does the township-day carry the relief flood flag. Both see the same fifteen features and the same chronological assignment of rows to train, validation, and test. Preprocessing for the histogram model is the feature construction in the training script. Preprocessing for the logistic model is median imputation and standardization fitted on the training rows only.

The selection metric for the histogram candidates is validation average precision, which is the precision-recall area. The threshold for either classifier is then chosen by flood-class F1 on the validation block. The test block is scored once the threshold exists. That order is the method behind the tables in Chapter 5.

3.1.1 Class-Weighted Logistic Regression (interpretable baseline)

The baseline is a logistic regression with balanced class weights, the liblinear solver, and a maximum of 2,000 iterations. Numeric columns pass through a median imputer and a standard scaler. The imputer and the scaler are fitted on the training block and saved inside the pipeline with the classifier.

The decision threshold is the validation threshold that maximizes flood-class F1. The saved threshold is 0.87. The file is marked as a secondary model, and the metadata names the histogram model as primary. Coefficients remain available for inspection because the classifier is linear after scaling. The prediction service does not load this file when it scores a township-day.

3.1.2 Histogram Gradient Boosting (served flood model)

The served classifier is sklearn.ensemble.HistGradientBoostingClassifier. Each candidate uses a learning rate of 0.05, up to 300 iterations, an L2 penalty of 1.0, early stopping, and a fixed random seed. Three candidates are fit on the training block: depth 3 with at least 20 samples in a leaf, the same tree with balanced class weight, and depth 4 with at least 10 samples in a leaf. The candidate with the highest validation precision-recall area is kept. The saved name is hgb-depth3.

An isotonic calibrator is fitted on the validation scores of that candidate and stored with the trees. The F1 search for the threshold runs on the calibrated validation probabilities and selects 0.05. Test probabilities are produced only after that choice. The metadata flag test_used_for_training_calibration_or_threshold is false.

The fifteen inputs are one-day, three-day, seven-day, fourteen-day, and thirty-day rainfall totals; temperature; relative humidity; wind speed; latitude; longitude; the sine and cosine of the day of the year; the previous day's modeled discharge; and the mean of lagged discharge over three days. Extended names such as soil moisture, runoff, CHIRPS rainfall, HAND, a 100-year depth grid, and a built-up floodplain fraction are listed in the metadata and are not part of the fifteen columns the classifier receives.

3.2 Application Services and External Data Clients

The HTTP service is a FastAPI application. Separate routers cover health, hydrology and the flood score, prediction timelines, allocation plans, inventory, operations, and a replay of archived days. The process loads the flood artifact once and refuses a score when the file or the metadata is missing, or when the artifact has no probability method.

External clients fill inputs the local table does not already hold. Open-Meteo supplies rainfall. GloFAS modeled discharge is read through the flood interface associated with that client. NASA POWER supplies retrospective temperature, humidity, and wind. OpenTopography supplies Copernicus elevation for display. Driving geometry, when the public routing service answers, comes from OSRM. If that call does not return a road path, the allocation code keeps a straight-line fallback and can also record a flood-aware corridor. Keys for services that require them stay in the environment and are not written into source listings or API responses.

3.3 Relief, Census, and Inventory Data Preparation

The daily flood table is validated before features are built. Validation checks the schema, the binary flood flag, duplicate township-date keys, coordinates, and non-negative rainfall and discharge. The approved daily file is copied into an immutable raw location and the processed copy used for training is validated_daily.csv. The label column remains the relief flag.

The 2024 Bago census extract supplies population by sex, urban and rural residence where the workbook publishes it, age groups for the region and the districts, and household

size and dwelling type. Age groups are not published for every township. Census figures are reference context for the allocation screen. They are not columns in the fifteen-feature flood model.

Inventory images are prepared per detector. Foods cover rice, oil, and canned fish. Clothing covers T-shirts, men's longyi, and women's longyi. Kitchenware covers bowl, knife, mug, spoon, soap, water dipper, plate, and pot. Utilities cover bed net, blanket, towel, and box. The catalog class fund has no detector. Class names in each data file match the item codes the scan screen expects.

3.4 Feature Construction and Numeric Representation

Feature construction sorts the daily table by township and date. Rainfall windows are rolling sums inside the township: the current day, and trailing windows of three, seven, fourteen, and thirty days that include the current day. A window that cannot be filled because the township series is too short stays missing, and rows that still miss any of the fifteen inputs are dropped. The served metadata records 638 such rows.

Modeled discharge is shifted by one day inside the township. The three-day discharge feature is the mean of that lagged series over three days. Seasonality is the sine and cosine of the day of the year on a 365.25-day circle. Latitude and longitude are the township coordinates stored on the daily row.

Temperature, humidity, and wind are joined from the NASA POWER cache. The training script may fill a NASA gap from the Open-Meteo archive. The saved metadata records zero values filled from that archive, so the served file was built from NASA meteorology on the rows that remained. Household counts and affected-people counts are impact fields. They are targets for a planning illustration, not flood-model inputs.

Feature	Construction
rainfall_1d_mm	Same-day rainfall total
rainfall_3d_mm	Three-day rolling rainfall sum, current day included
rainfall_7d_mm	Seven-day rolling rainfall sum, current day included
rainfall_14d_mm	Fourteen-day rolling rainfall sum, current day included

rainfall_30d_mm	Thirty-day rolling rainfall sum, current day included
temperature_c	NASA POWER 2 m temperature
relative_humidity_pct	NASA POWER relative humidity
wind_speed_m_s	NASA POWER 2 m wind speed
latitude, longitude	Township coordinates
day_of_year_sin, day_of_year_cos	Seasonal position on a 365.25-day circle
discharge_lag_1d_m3s	Modeled discharge shifted by one day
discharge_lag_3d_mean_m3s	Three-day mean of the lagged discharge

3.5 Application Database and Processed Data Store

Two kinds of storage are kept apart. Model files, the validated daily CSV, the split assignment, the census extract, and the offline hydrology cache stay on the filesystem. The API reads the flood joblib and its JSON metadata by path. It does not place those artifacts inside the application database.

The application database holds operational records: locations, plans, alerts, and inventory rows. Local development uses SQLite through the default database URL. The Docker stack uses PostgreSQL with PostGIS for the same role. A second SQLite file under data/offline can hold merged hydrology for hindcast dates so the map can read stored rainfall and discharge without a live call.

This store answers lookups by township and date and stores application rows. It does not embed text or rank passages. The flood decision remains a classifier over the fifteen numeric columns.

3.6 Multi-Source Hydrometeorological Inputs

No single archive supplies every column. Rainfall in the training table is Open-Meteo reanalysis already stored on the validated daily file. Discharge is GloFAS river discharge stored from the Open-Meteo flood interface. Humidity is NASA POWER RH2M. Temperature and wind are NASA POWER T2M and WS2M.

For a live outlook the feature builder assembles the same fifteen names. Where a retrospective NASA value exists it is preferred for temperature, humidity, and wind. Short

gaps in the local hydrology cache can be carried forward from the last stored rainfall, discharge, or meteorology so the screen can still draw a day. A carried-forward value is a stored neighbour, not a new measurement, and the interface is still a prototype view.

Elevation from Copernicus is requested for the selected point and shown as terrain context. The 2024 census extract is requested as population context. Neither source is concatenated onto the fifteen flood features.

3.7 Leakage Control and Claim Limits

The split file is the authority for which rows may be fit. The trainer merges it onto the engineered table and stops if the dates are out of chronological order. Thresholds and the isotonic calibrator see validation rows only. The published test metrics are computed with that frozen threshold.

Same-day rainfall is contemporaneous with a historical label. It is a feature of the recorded day, not a lead time. Discharge features are lagged, which keeps the same-day discharge value out of those two columns. Impact counts stay out of the flood feature list. Census totals stay out of the flood feature list.

Claim limits are part of the method. The API description states that the service is an academic flood-risk prototype and is not for operational warnings. Risk bands from the heuristic in the server are a documented formula, not a third learned model. Assistance priority taken from households and affected people uses a retrospective score and is not a forecast severity.

3.8 Request Handling and Response Generation

A prediction request names a township and a date. The service builds the fifteen features, checks that each required name is present, and calls predict_proba on the loaded histogram model. The flood class is positive when the probability is at least 0.05. The response carries the probability, the class, the threshold, and the model version read from the metadata.

The academic risk score is a bounded sum of rainfall, modeled discharge, and the model value. The score is capped at 100. Bands are low below 25, moderate below 50, high below 75, and severe from 75. A separate display band groups the probability itself at 0.25, 0.50, and 0.75. These bands explain the colour on the map. They do not replace the precision and recall of the held-out test.

An assistance request prefers a relief-workbook impact for that township and date when one exists. Otherwise it can read an archived simulation day. Otherwise it scales a relief or census baseline by the flood probability, with an eight percent census fallback when the relief baseline is empty, and caps the headcount at the township population. Shelter kits, three-day food rations, and drinking water are the fixed ratios named above. A learned two-stage help artifact is used only when that file is present on disk. It is not in the artifact directory of this tree, so the screens follow the workbook, archive, or scaled-baseline path.

Inventory detection accepts an image or a video, runs the detector for the requested goods class, and returns boxes that the user can confirm into stock. Allocation can then compare route options and store the selected plan.

3.9 Proposed System Workflow

The workflow is a sequence of stored files and request-time calls. It is the path a user walks from the home screen to a flood day, a plan, and a stock count.

First the daily table is validated and the fifteen features are built inside each township. Second the histogram candidates are fit on the training dates, one candidate is chosen on validation average precision, the calibrator and the threshold are fit on validation, and the test block is scored and stored in metadata. The logistic pipeline is fit on the same split and saved beside the served file. Third the API loads the histogram artifact. Fourth a map request builds features for the selected township and date and returns the probability, the class, and the risk band. Fifth an allocation request turns workbook, archive, or scaled census figures into a three-day package and a route among centroids. Sixth an inventory request counts visible goods and, after confirmation, updates stock.

Chapter 4

System Design and Implementation

The implementation is divided into a machine-learning tree, a FastAPI service, a React interface, and Docker packaging. Training writes files. The service reads files and answers HTTP. The interface renders those answers and labels prototype inputs.

4.1 Overview of the Proposed System

HydroFlow V2 has four screens: Home, Flood Prediction, Resource Allocation, and Inventory Management. The flood screen is a Leaflet map of Bago districts and townships, a

timeline, forecast cards, a hazard panel, historical weather, and recorded flood history. The allocation screen lists township plans, compares routes, and can open a print preview. The inventory screen scans a photograph or a video and confirms counts into a regional stock list.

The backend process exposes health, hydrology, prediction, allocation, inventory, operations, and simulation routes. High-level prediction paths include a timeline, map scores, and a region forecast, together with the earlier predict and outlook calls. Allocation covers stores, plans, confirmation, and dispatch records. Inventory covers detection, the catalog, and stock entries.

4.2 Scope of the Project

The geographical scope is twenty-three townships in Bago Region: Bago, Daik-U, Htantabin, Kawa, Kyauktaga, Madauk, Minhla, Monyo, Okpho, Oktwin, Pantaung, Pyay, Pyu, Shwedaung, Shwegyin, Taungoo, Thanatpin, Thayarwady, Thegon, Waw, Yedashe, and Zigon. The flood model scores a township-day. It does not draw a street-level inundation polygon as its learned output.

The functional scope covers the served histogram model, the logistic comparison file, the risk heuristic, the assistance illustration, centroid routes, and four goods detectors. Official warning products and job records exist as operational endpoints for provenance and replay. They do not convert the prototype into an agency warning desk.

Out of scope are public dissemination, measured gauge height, a complete flood inventory, and any use of the scores as a sole basis for evacuation or procurement. An extension that needs those functions also needs new labels and an authority outside this codebase.

4.3 Dataset Description

The flood dataset is a panel of township-days. After feature construction, 39,534 rows have every served input and the flood flag. The chronological validation block holds 8,030 of them, including 167 flood days. The chronological test block holds 8,052, including 11 flood days. The label text stored with the model is an observed relief flood flag, with no synthetic flood rows added.

Detection datasets are separate image collections under the inventory raw tree, built into YOLO layouts for foods, clothing, kitchenware, and utilities. Their metrics in Chapter 5 are the last recorded training epoch in each detector history. They are not rows of the flood table.

4.3.1 Source Data Files

The validated daily table is data/processed/validated_daily.csv. The chronological assignment is data/processed/split_assignments.csv. NASA humidity is data/external/nasa_power_daily.csv, with temperature and wind caches beside it. The census workbook is the 2024 Bago extract, and the service reads the structured JSON copy. Township centroids and administrative names come from the admin boundary files used by the map.

The served flood weights are ml/models/bago_flood_model.joblib with bago_flood_model_metadata.json. The logistic comparison is bago_flood_model_logistic.joblib with its metadata. Detector weights, when promoted, live under ml/models/object_detect for foods, clothing, kitchenware, and utilities, each with a training_history.csv copied from the training log.

Offline hydrology for hindcast dates can be read from data/offline/flood_sources.sqlite. Relief workbooks remain the source of the flood flag and of impact counts when a planning date matches a workbook row.

4.3.2 Data Preprocessing

Preprocessing for the flood model is deterministic and township-wise, as Chapter 3 describes. Rows with a missing feature or a missing flag are removed rather than filled inside the histogram trainer. The logistic pipeline is the exception that imputes the median, and it imputes from the training block only.

The split is not redrawn at serving time. Both models consume the saved assignment. A check in the trainer rejects a file whose training dates do not fall entirely before validation, or whose validation dates do not fall entirely before the test.

Image preprocessing follows the detector trainer: class lists are built from the raw photo folders, names are aligned to item codes, and the training entry point records precision, recall, and mean average precision for each epoch. Promotion copies the chosen weights into the model directory the API resolves first.

4.4 Technical Stack and Configuration

The service is Python with FastAPI and Uvicorn. Tables in memory use pandas. The flood models use scikit-learn and are stored with joblib. The application database uses

SQLAlchemy, with SQLite locally and PostgreSQL through psycopg in Docker. Image detection uses OpenCV and Ultralytics. Tests use pytest.

The interface is React 19 with TypeScript and Vite. Maps use Leaflet. Plan sheets can be exported with the PDF helpers in the frontend. Interface tests use Vitest. The browser build is served on port 8080 in Docker and on the Vite port during local development.

Docker Compose defines three services. The database image is PostGIS 16. The API container depends on a healthy database and reads the validated daily file from a read-only mount. The frontend container depends on a healthy API and publishes port 8080. API keys and the database URL are environment settings. The compose file supplies a development database password for the local stack; a deployment outside this prototype would replace it.

Layer	Role in this prototype
Python, FastAPI, Uvicorn	HTTP service and request validation
pandas, scikit-learn, joblib	Features, classifiers, saved artifacts
SQLAlchemy, SQLite, PostgreSQL/PostGIS	Operational records
OpenCV, Ultralytics	Relief-goods detection
React, TypeScript, Vite, Leaflet	Map and planning screens
Docker Compose	Database, API, and frontend

4.5 System Implementation

Training of the served flood model is the script that reads the validated daily table, joins NASA meteorology, engineers the fifteen columns, and writes the joblib and the metadata. The logistic script states that it scores the held-out block once and does not replace the histogram file. The API path resolves ml/models/bago_flood_model.joblib unless an environment path overrides it.

On startup the API initializes the operational database and ensures a demonstration stock so the inventory screen is not empty. CORS allows the local Vite origin and the Docker origin on port 8080. Each router returns typed responses. A missing model becomes an unavailable-model error rather than a silent score of zero from an untrained default, except where an assistance path explicitly records a fallback version name.

The flood screen requests calendar bounds, map scores, and a timeline. The allocation screen creates a plan, compares route options for a township row, and stores the selected option. Route methods recorded in the service include an OSRM road path, a straight-line fallback, and a flood-aware corridor. Stops are township centroids, so the line is an illustration of order and distance, not a site survey.

The inventory service maps detector class names onto item codes, resolves weights from the promoted directory, and can confirm a detection into a stock entry. Video detection uses the same class list. The user confirms a count before it is treated as stock on hand.

Chapter 5

Results and Discussion

This chapter reports the scores stored with the served model and with the logistic comparison, then describes what the screens show. Detection histories are reported in their own table because they measure boxes on images, not flood days.

5.1 Introduction

All flood figures in this chapter are taken from the JSON metadata written when the models were trained. They are not a fresh fit performed for the book. The test block was not used to train, to calibrate, or to pick the threshold. Validation figures are included so the threshold choice is visible, and they are labeled as validation.

The test set contains eleven flood days. With so few positives, a change of one true positive moves recall by about nine percentage points. The tables are therefore a description of this holdout, not a stable estimate of performance on a future monsoon.

5.2 System Output and Functional Results

A successful health check shows that the API process is up. A flood request returns a probability, a binary class at the saved threshold, the threshold itself, and the model version. For the served file that version name resolves to the histogram model. Map scores colour townships from the same response family. Historical weather and flood history are context panels. They replay stored rows and are not an extra trained model.

An allocation result lists expected people and households, an assistance priority, shelter kits, three-day food rations, and drinking water, together with a route method. When the

numbers come from a workbook row, the version string says so. When they come from the flood probability and a baseline, the version string records that fallback. An inventory result lists class counts and boxes. Stock changes after the user confirms.

These outputs are functional in the sense that the screens and the service implement them. Functional here does not mean that the flood class is precise. The next sections give the holdout counts that bound that claim.

5.3 Logistic Regression Baseline versus Served Histogram Gradient Boosting

Both models see 8,052 untouched test rows and the same eleven flood days. The histogram model uses threshold 0.05. The logistic model uses threshold 0.87. The histogram model records precision 0.0113, recall 0.6364, F1 0.0222, precision-recall area 0.0085, and ROC area 0.8684. The logistic model records precision 0.0025, recall 0.0909, F1 0.0048, precision-recall area 0.0063, and ROC area 0.7950.

The confusion counts make the same comparison concrete. The histogram model marks 7 of the 11 flood days and also marks 612 non-flood days as flood, with 4 flood days missed and 7,429 non-flood days left negative. The logistic model marks 1 of the 11 flood days and marks 401 non-flood days as flood, with 10 flood days missed and 7,640 non-flood days left negative.

Accuracy is 0.9235 for the histogram model and 0.9490 for the logistic model. A prediction of no flood on every test row reaches accuracy 0.9986. The logistic model is closer to that rule because its threshold is high and its recall is one flood day in eleven. The histogram model recovers more of the eleven days and pays for that recall with more false alarms. Precision-recall area, which was also the criterion that chose the histogram candidate, is higher for the served model than for the logistic baseline on this block.

On the validation block, which was used to choose the histogram threshold, the served model sees 8,030 rows and 167 flood days. At threshold 0.05 it records precision 0.0701, recall 0.2874, F1 0.1127, and precision-recall area 0.0624, with 48 true positives and 637 false positives. Validation is easier than the test block because the positives are more numerous. It is not a substitute for the untouched test.

Model	Threshold	Precision	Recall	F1	PR-AUC	ROC-AUC

Histogram gradient boosting	0.05	0.0113	0.6364	0.0222	0.0085	0.8684
Logistic regression	0.87	0.0025	0.0909	0.0048	0.0063	0.7950

Model	True negative	False positive	False negative	True positive
Histogram gradient boosting	7,429	612	4	7
Logistic regression	7,640	401	10	1

5.4 Graphical User Interface

The shell is a single page with four destinations. Home introduces the workspace. Flood Prediction centres a Leaflet map and places the timeline, summary cards, forecast cards, hazard depth, and historical context beside it. Resource Allocation lists township rows for a plan, opens a comparison of route options, draws the selected path, and can preview a printable sheet. Inventory Management shows catalog categories, a scan action for a still image or a video, and the stock that remains after confirmation.

The map uses township boundaries and centroids already served by the API. Colours follow the risk and probability bands defined in the service. Text on the planning views names the prototype role of the counts. The route line is the geometry returned for the chosen method. A user can switch among the screens without a second login step because the prototype has no accounts.

5.5 System Testing and Model Evaluation

Evaluation of the flood classifiers is the chronological protocol in Chapter 3. Fit on train, select the histogram candidate and both thresholds on validation, and publish the test metrics from the metadata. The majority-class accuracy is stored next to those metrics so a high accuracy cannot be misread.

Software tests cover the service and the interface. Backend tests exercise health, prediction, assistance fallbacks, allocation, and the mapping from detector names to item codes. Machine-learning tests cover the feature and training helpers under the ml tree.

Frontend tests render the dashboards and the allocation interactions with Vitest. The commands live in the verification note: pytest for the Python trees, and npm test plus a production build for the interface.

Detector evaluation in this book is the final epoch written in each training_history.csv. Those columns are the validation precision, recall, and mean average precision logged during training. They are not an untouched flood test, and they are not a count of goods in a warehouse audit.

Detector	Final epoch	Precision	Recall	mAP50	mAP50-95
Foods	52	0.831	0.912	0.875	0.704
Clothing	41	0.910	0.882	0.965	0.656
Kitchenware	80	0.936	0.958	0.942	0.756
Utilities	71	0.846	0.898	0.887	0.737

5.6 Results Analysis

The served model finds seven of the eleven test flood days. That recall is the strongest flood-class result in the comparison, and the precision of 0.0113 means that a positive mark is usually a non-flood day. A planner who treated every positive mark as a confirmed flood would be responding to hundreds of quiet days. The prototype therefore shows the probability and the band, and the book keeps the false-positive count beside the true-positive count.

The logistic baseline is more specific in the weak sense that it raises fewer false alarms, and it misses ten of the eleven flood days. Its higher accuracy follows from predicting the majority class more often. For a rare relief flag, that accuracy repeats the base rate. The precision-recall area remains below 0.01 for both models on the test block, which is the quantitative statement that ranking quality on this holdout is poor.

The validation block, with 167 flood days, supports a higher F1 for the served model than the test block does. The drop from validation F1 0.1127 to test F1 0.0222 is consistent with a shift in how rare and how placed the later positives are. Eleven test positives cannot identify which township drives the drop. The training notes already suppress township test metrics when a township has fewer than five test flood days, and no township in this design has five.

The detector table is stronger than the flood table, and it answers a different question. Kitchenware, clothing, foods, and utilities all end training with validation mAP50 above 0.87 on the logged epoch. A high mAP50 means the boxes agreed with the labeled images in that training run. It says nothing about the relief flood flag.

5.7 Model Selection and Justification

The served flood model is the histogram candidate hgb-depth3 because it won the validation precision-recall comparison among the three candidates the trainer fit, and because the saved logistic comparison also records the histogram model as the better of the two on precision-recall area. The logistic pipeline remains in the tree as the interpretable baseline. It is the model a reader can inspect through scaled coefficients. It is not the file the prediction path loads.

The threshold 0.05 is low because flood-class F1 on the validation block preferred it. A higher threshold would resemble the logistic operating point: fewer alarms and fewer of the rare flood days. The book reports the operating point that the training rule actually saved.

An earlier twelve-feature logistic study removed NASA soil wetness and humidity so those retrospective fields would not be required on an outlook date. Its untouched-test flood F1 was about 0.029. That study used a different feature list from the fifteen columns in the current metadata. It is cited here so the two results are not exchanged. The justification for serving the histogram file is the fifteen-feature comparison above, together with the fact that the API path points at that file.

No detector replaces the flood model. Goods detection is selected per catalog class because each training set has its own item list. Fund remains a manual catalog entry.

5.8 Advantages of the Proposed System

The prototype puts the township map, the flood score, the three-day illustration, the route, and the stock count in one interface backed by one service. A reader can see which file produced the score because the response carries the model version and the metadata carries the threshold and the test counts.

The evaluation design is visible. The split is chronological. The test flag is stored. The majority-class accuracy is stored. The logistic baseline uses the same rows, so the gain in recall is measured against a stated alternative rather than against an unpublished run.

Local files allow the map to read stored hydrology and the census extract when a live provider is slow or absent, within the limits of those files. Demonstration stock lets the inventory path be reviewed without a prior warehouse import. These are advantages for study and for a supervised demonstration.

Chapter 6

Conclusion and Future Scope

HydroFlow V2 is a complete academic path from a Bago daily table to a served flood score and a set of planning screens. The path is documented with the holdout counts that bound its skill.

6.1 Summary of the Project

The project builds and serves a histogram gradient boosting classifier for a relief flood flag on twenty-three townships. Fifteen features cover rainfall windows, NASA meteorology, coordinates, seasonality, and lagged modeled discharge. The saved candidate is hgb-depth3. The threshold is 0.05. On the untouched test block the model reaches recall 0.6364 and precision 0.0113, with 7 true positives and 612 false positives out of 11 flood days and 8,052 rows.

A class-weighted logistic regression on the same features is the baseline, with threshold 0.87, recall 0.0909, and precision 0.0025. The interface adds an assistance illustration, centroid routes, and four goods detectors. The book records the last-epoch detection scores separately from the flood scores.

6.2 Limitations

The relief flag is an incomplete label. Precision on the untouched test is 0.0113. Eleven test flood days cannot support a township ranking or an operational probability. The scores are not independently calibrated for public warning. Isotonic calibration was fit on the validation block and was not given a later calibration set.

GloFAS discharge is modeled discharge, not a gauge height. Rainfall in the training table is reanalysis. Carried-forward hydrology fills gaps with the last stored value. Copernicus elevation includes canopy and buildings and is display context only. Census age groups are not published for every township.

Assistance counts are illustrations from a workbook row, an archived day, or a baseline scaled by the flood score and capped by population. The fixed ratios are not a needs assessment. Routes follow township centroids. Alerts stored by the application are prototype records. Detector scores come from training logs. The class fund is not detected. The earlier twelve-feature result must not be quoted as the score of the file the API loads.

6.3 Future Extensions

The data extension that would change the flood scores is a set of independently verified flood labels for later years, with event dates and township provenance kept, followed by a new untouched time block. Synthetic positives would not serve that purpose.

A scientific extension is to pair the relief flag with gauged stages and a local rainfall network, and to publish reliability of the probabilities on a block that was not used for the threshold. A planning extension is to replace centroid illustrations with surveyed sites and stock that has been counted outside the demonstration seed. Any of those steps still leaves public use dependent on review by the authorities who issue warnings.

References

[1] Nevo, S. et al., 2022. Flood forecasting with machine learning models in an operational framework. Hydrology and Earth System Sciences, 26, pp.4013–4032.

[2] Mosavi, A., Ozturk, P. and Chau, K.W., 2018. Flood prediction using machine learning models: Literature review. Water, 10(11), p.1536.

[3] Breiman, L., 2001. Random forests. Machine Learning, 45(1), pp.5–32.

[4] Friedman, J.H., 2001. Greedy function approximation: a gradient boosting machine. Annals of Statistics, 29(5), pp.1189–1232.

[5] Pedregosa, F. et al., 2011. Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, pp.2825–2830.

[6] Kratzert, F., Klotz, D., Brenner, C., Schulz, K. and Herrnegger, M., 2018. Rainfall–runoff modelling using Long Short-Term Memory (LSTM) networks. Hydrology and Earth System Sciences, 22, pp.6005–6022.

[7] Harrigan, S., Zsoter, E., Alfieri, L., Prudhomme, C., Salamon, P., Wetterhall, F., Barnard, C., Cloke, H. and Pappenberger, F., 2020. GloFAS-ERA5 operational global river discharge reanalysis 1979–present. Earth System Science Data, 12, pp.2043–2060.

[8] Open-Meteo, n.d. Weather and flood APIs. Available at: https://open-meteo.com/.

[9] NASA Langley Research Center, n.d. Prediction of Worldwide Energy Resources (POWER). Available at: https://power.larc.nasa.gov/.

[10] European Space Agency, n.d. Copernicus DEM. Copernicus Data Space Ecosystem.

[11] Kaufman, S., Rosset, S., Perlich, C. and Stitelman, O., 2012. Leakage in data mining: Formulation, detection, and avoidance. ACM Transactions on Knowledge Discovery from Data, 6(4), pp.1–21.

[12] Saito, T. and Rehmsmeier, M., 2015. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. PLoS ONE, 10(3), e0118432.

[13] Sphere Association, 2018. The Sphere Handbook: Humanitarian Charter and Minimum Standards in Humanitarian Response. 4th ed. Geneva: Sphere Association.

[14] Redmon, J., Divvala, S., Girshick, R. and Farhadi, A., 2016. You Only Look Once: Unified, real-time object detection. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp.779–788.

[15] Jocher, G., Chaurasia, A. and Qiu, J., 2023. Ultralytics YOLO. Available at: https://github.com/ultralytics/ultralytics.